

Semana 11 — Búsqueda Lineal y Binaria · HITO 2

COMPRENDE — Lineal vs Binaria: el costo de buscar

Reto IA — Comprende: El invariante bajo la lupa

Arreglo correctamente ordenado

El equipo afirma: “si el objetivo existe, siempre lo encontramos”. Pregunta: ¿cómo demuestra el invariante que en cada iteración el objetivo sigue dentro del rango [izq, der]? No me respondan con intuición, sino con la propiedad que se mantiene.

(Cuando contesten, yo les preguntaré: ¿qué pasaría si el rango se reduce mal y el objetivo queda fuera?)

2. Error de orden en un solo elemento

Pregunta: si un elemento está fuera de lugar, ¿el invariante “el objetivo está en el rango” sigue siendo válido?

(Cuando contesten, yo les preguntaré: ¿cómo detectaría el algoritmo que el invariante se rompió, o simplemente falla sin aviso?)

3. Iterativo vs recursivo

Pregunta: ¿la versión recursiva mantiene exactamente la misma invariante que la iterativa, o cambia la forma de expresarlo?

(Cuando contesten, yo les preguntaré: si cambia la formulación, ¿qué parte del razonamiento se conserva idéntica en ambas versiones?)

IA-REFLEXION-C: El equipo descubrió que el invariante no es “siempre encontramos el objetivo”, sino que el objetivo, si existe, permanece dentro del rango activo en cada paso. La condición que garantiza corrección es que el arreglo esté ordenado; si se rompe el orden, el invariante deja de ser válido y la búsqueda puede fallar.

APLICA — Implementar, medir y ensamblar el HITO 2

Reto 5 — Búsqueda lineal + binaria + benchmark

Una oración sobre lo que debe garantizar

La línea de cálculo de mid debe asegurar que el índice central siempre esté dentro del rango [izq, der], preservando el invariante de que el objetivo, si existe, permanece en ese rango.

2. La línea correcta (solo esa)

python

```
mid = (izq + der) // 2
```

3. Pregunta socrática para el equipo

Si usamos exactamente esa fórmula, ¿por qué el índice mid nunca se sale del rango activo y cómo eso mantiene vivo el invariante de corrección en cada iteración?

IA-REFLEXION-A: El equipo necesitó ayuda en la línea de cálculo de mid, para entender cómo preserva el invariante. Implementaron sin asistencia el bucle while $izq \leq der$ y la actualización correcta de izq y der.

Escenario: El algoritmo encuentra una ocurrencia del puntaje buscado con búsqueda binaria. Luego expande hacia la izquierda y hacia la derecha para recolectar todas las películas con ese mismo puntaje.

Pregunta inicial: ¿qué debe garantizar el invariante en cada paso de la expansión para que no se pierda ninguna película con el puntaje buscado?

(Cuando el equipo responda, yo les preguntaré: si el invariante se rompe en un lado, ¿cómo afectaría la corrección del resultado final?)

Pregunta de seguimiento: ¿la expansión hacia izquierda y derecha mantiene la misma invariante, o hay que formularlo de manera distinta para cada dirección?

IA-REFLEXION-A (continuación): El equipo descubrió que el cálculo de mid requería apoyo, pero implementó por sí mismo el bucle y las actualizaciones de izq/der. Ahora, en la expansión de duplicados, comprendieron que el invariante debe garantizar que cada paso incluye todas las ocurrencias adyacentes sin omitir ninguna.

Tamaño n	Búsqueda lineal ($O(n)$)	Búsqueda binaria ($O(\log_2(n))$)	Factor teórico $n/\log_2(n)$
1,000	~1,000 pasos	~10 pasos	~100×
10,000	~10,000 pasos	~14 pasos	~714×
100,000	~100,000 pasos	~17 pasos	~5,882×
1,000,000	~1,000,000 pasos	~20 pasos	~50,000×

Interpretación para el equipo:

- La búsqueda lineal escala directamente con n.
- La búsqueda binaria crece muy lentamente porque depende de $\log_2(n)$.
- El factor teórico muestra cuántas veces más eficiente debería ser la binaria frente a la lineal.

REFLEXIONA-1/2/3 (adelanto): El equipo podrá comparar estos valores con los tiempos reales y observar que la aceleración práctica nunca alcanza exactamente el factor teórico, pero sí confirma la ventaja de la búsqueda binaria en catálogos grandes.

Reto 6 — Búsqueda binaria con variantes: primera y última ocurrencia

Variable para recordar la posición

Necesitamos una variable candidata (por ejemplo pos) que guarde el índice cada vez que encontramos el objetivo, para no perderlo aunque sigamos buscando más a la izquierda.

2. Actualización de $der = mid - 1$

No estamos descartando la posición encontrada, porque ya la guardamos en pos; lo que hacemos es forzar al algoritmo a seguir explorando hacia la izquierda en busca de una ocurrencia anterior.

3. Garantía al terminar el bucle

El invariante asegura que cada vez que encontramos el objetivo, lo registramos, y al terminar el bucle la última posición guardada en pos corresponde exactamente a la primera ocurrencia.

IA-REFLEXION-B: El equipo entendió que la variante no retorna de inmediato, sino que guarda la posición y sigue buscando en la dirección adecuada. Esto permite localizar la primera o última ocurrencia en $O(\log n)$ incluso con duplicados, evitando la expansión lineal.

REFLEXIONA — Defender el invariante y el pipeline

Reto IA — Reflexiona: Simulacro de defensa oral con foco en invariantes

D1 — Traza sobre [2, 4, 6, 8, 10, 12, 14] buscando 10

Respuesta del equipo (ejemplo): “Comenzamos con $mid=6$ (índice 3, valor 8). Como $10 > 8$, movemos $izq=4$. Luego $mid=10$ (índice 4), encontramos el objetivo y retornamos.” Evaluación: 100%. La traza está completa y correcta.

D2 — Invariante con $izq = mid$

Respuesta del equipo (ejemplo): “El invariante asegura que el objetivo está en $[izq, der]$. Si actualizamos $izq=mid$, todavía lo mantenemos.” Evaluación: 60% (rescate). La explicación es incompleta: no se justifica por qué $izq=mid$ puede romper el invariante. Rescate nivel 2: “¿Puedes decir en una sola oración qué hace búsqueda binaria?” Respuesta del equipo (ejemplo): “Divide el rango en mitades y mantiene al objetivo dentro del rango activo.” Evaluación final: Rescate aplicado, concepto correcto.

D4 — HITO 2: Justificación del pipeline completo

Respuesta del equipo (ejemplo): “El pipeline ordena el catálogo con el algoritmo elegido en S10, luego usa búsqueda binaria para localizar puntajes y buscar_rango para duplicados. Así cumple el requerimiento de la Ing. Sofía.” Evaluación: 100%. La justificación es clara y conecta cada parte del pipeline.

IA-REFLEXION-R: El ítem más difícil fue D2, porque el equipo confundió la actualización de límites con la preservación del invariante. Necesitan repasar cómo las condiciones de actualización ($izq = mid + 1$, $der = mid - 1$) garantizan que el objetivo permanezca dentro del rango activo.

VALIDA — Casos de prueba y verificación del HITO 2

Reto IA — Valida: Casos adversariales para búsqueda binaria y el pipeline

Paso 1 — Declaración de invariantes

Equipo, antes de ejecutar, declaren explícitamente los 3 invariantes que su búsqueda binaria garantiza (ejemplo: “el objetivo, si existe, está en $[izq, der]$ ”, “mid siempre está dentro del rango”, “el rango se reduce en cada iteración”).

Paso 2 — Casos adversariales

1. Arreglo de longitud par Caso: $[1, 2, 3, 4, 5, 6]$, objetivo = 4. Predigan la salida antes de ejecutar. (Este caso puede romper el invariante de cálculo de mid si hay error off-by-one en arreglos pares.)
2. Objetivo en el primer elemento Caso: $[7, 8, 9, 10]$, objetivo = 7. Predigan la salida antes de ejecutar. (Este caso prueba si el invariante “el objetivo está en el rango” se mantiene cuando el objetivo está en el extremo izquierdo.)
3. Objetivo en el último elemento Caso: $[11, 12, 13, 14]$, objetivo = 14. Predigan la salida antes de ejecutar. (Este caso prueba el extremo derecho y puede revelar errores en la actualización de izq.)
4. Arreglo de 2 elementos Caso: $[20, 30]$, objetivo = 30. Predigan la salida antes de ejecutar. (Este caso es crítico: si izq y der no se actualizan correctamente, el invariante de reducción de rango puede romperse.)

Paso 3 — Ejecución y validación

El equipo ejecuta cada caso y compara.

- Si hay discrepancia, yo señalo: “El invariante X falló en la iteración Y — ¿por qué?”
- No doy el fix, solo marco la propiedad rota.

IA-REFLEXION-V: El equipo encontró que el caso más revelador fue el arreglo de 2 elementos, donde un error off-by-one rompió el invariante de reducción de rango. El pipeline pasó los demás casos, pero necesita repasar cómo las actualizaciones de límites preservan los invariantes en extremos y arreglos pequeños.

PROFUNDIZA — Límites de la búsqueda y puente a Tablas Hash

Reto IA — Profundiza: Del $O(\log n)$ al $O(1)$ — motivación para tablas hash

La función mágica $f(\text{título})$

Imaginen que existe una función que, dado el título de una película, devuelve directamente su posición en el catálogo. Pregunta: ¿qué propiedades debería tener esa función para que siempre

funcione? (pista: debe ser determinista y consistente, no puede dar posiciones distintas para el mismo título).

2. Colisiones abiertas

Ahora supongan que dos títulos distintos producen el mismo índice. Pregunta: ¿qué pasaría con la corrección del pipeline si dos películas diferentes “chocan” en la misma posición? (Dejen esta pregunta abierta: no resuelvan todavía cómo manejar colisiones, eso será tema de S12).

3. Orden vs desorden

Pregunta: ¿esta función mágica necesita que el catálogo esté ordenado, como en búsqueda binaria? ¿Por qué sería revolucionario que ya no dependiera del orden?

IA-REFLEXION-P: Al equipo le queda pendiente entender cómo manejar las colisiones cuando dos películas distintas generan el mismo índice en la tabla hash. Esa es la pregunta que motivará la exploración en S12.